DHAR

Volume I — Vision & Specification

SPEC1

“A rule for every word. Nothing left to infer twice.”

Document	Spec1 — Language Specification
Version	3.1
Date	September 6, 2026
Status	Ratified
Part of	The Canon of Dhar, Volume I
Depends on	Spec0 — Overview & Vision
Author & Project Lead	Abdullahi Baba Garba

Preface: Scope and How to Read This Specification

Spec1 is the single, ratified source of truth for Dhar's syntax and semantics. Version 3.0 is a substantial expansion over 2.0, closing gaps identified by checking Dhar's surface against systems programming (C, Rust), object orientation (Java), and the ergonomics that make Python and JavaScript comfortable for web, mobile, and scripting work. Every keyword, type, and form defined here is a requirement for any implementation that calls itself Dhar. Where a construct's exact edge behavior is not yet pinned down, this specification makes a decisive, explicit choice and marks it as a **specification default**, open to future amendment through the process in the Master Index, rather than leaving a gap.

1. Notation & Conventions

Grammar fragments use a small EBNF-style notation:

Symbol	Meaning
`::=`	is defined as
`|`	or (alternation)
`[ x ]`	x is optional
`{ x }`	x repeats zero or more times
`'x'`	the literal token x
`NAME`	a token class defined elsewhere in this document

must/**must not** denote hard requirements; **should**/**should not** denote strong defaults; **may** denotes a permitted option — per the Master Index §3.1.

2. Lexical Structure

2.1 Source encoding

Dhar source files are UTF-8 text and **must** use the `.dhar` extension.

2.2 Identifiers

```
IDENT ::= ( LETTER | '_' ) { LETTER | DIGIT | '_' }
```

Identifiers are case-sensitive. The 46 reserved words in Appendix A must not be used as identifiers.

2.3 Comments

A # begins a line comment that runs to the end of the physical line. Dhar has no block-comment form *(specification default: deferred to Stage-delta's formatter/tooling design)*.

2.4 Literals

Kind	Form	Example
Integer	decimal, or 0x / 0b prefixed	`42`, `0xFF`, `0b1010`
Float	digits, one decimal point	`3.14`
String	double-quoted, escapes with backslash	`"Dhar"`
Interpolated string	f-prefixed, `{expr}` substitutes inline	`f"Hi, {name}"`
Character	single-quoted, one Unicode scalar or escape	`'D'`, `'\n'`
Boolean	the words true / false	`true`

2.5 Indentation & blocks

Dhar is indentation-delimited, in the tradition of Python. A line ending in `:` opens a block; every statement in it is indented one unit further than the line that opened it. Tabs and spaces must not be mixed within one file. mold, forge, alloy, and bond bodies (§9, §11, §12) use `{…}` instead — the deliberate exception marking data/structure/contract declarations apart from ordinary flow of control.

2.6 Statement separators

A `;` may separate two statements on the same physical line; never required, since a newline serves the same purpose.

3. Program Structure & Entry Point

A Dhar program is one or more `.dhar` files. Exactly one file **must** define a parameterless task named `core`; execution begins there. Cross-file dependencies use `pull`/`expose` (§17).

```
task core():
    gen("hello, dhar")
```

4. The Type System

4.1 Primitive types

Type	Width	Notes
i8, i16, i32, i64	8/16/32/64-bit	Signed integers, two's complement
u8, u16, u32, u64	8/16/32/64-bit	Unsigned integers
f32, f64	32/64-bit	IEEE 754 floating point
bool	1 byte (logical)	true or false
char	Unicode scalar	One character; distinct from a length-1 str
str	pointer + length	Immutable text; not null-terminated at the language level
raw[N]	N bytes	Fixed-size, uninterpreted memory buffer — see §14.4

4.2 User-defined types

mold (§9), alloy (§11), and bond (§12) each introduce named types. Collection types (§13) extend this vocabulary without their own keywords.

4.3 Static type inference

1. An integer literal with no fractional part infers to `i32`, unless later reassigned outside its range, requiring an explicit wider type.
2. A literal with a decimal point infers to `f64`.
3. A double-quoted or f-prefixed literal infers to `str`; a single-quoted literal infers to `char`.
4. `true`/`false` infer to `bool`.
5. A bracketed, braced, or parenthesized literal infers to the appropriate collection type (§13).
6. A binding's inferred type is fixed at declaration and does not change on reassignment, unless the binding is `wild` (§4.4).

4.4 Dynamic typing: wild

`wild` marks a binding, parameter, or return type as dynamically typed: its type may differ on every assignment, checked at runtime instead of compile time. A specification default trading Java-class compile-time guarantees for Python-class flexibility exactly where asked for.

```
x: wild = 5
x = "now a string"      # legal: x is wild
```

Prefer `alloy` (§11) over `wild` whenever the set of possible shapes is known in advance — alloy gives the same flexibility with compile-time exhaustiveness checking. wild is for genuinely unknown-until-runtime shapes, such as parsed external data.

4.5 Generics

A mold, alloy, bond, or task may be parameterized over one or more type placeholders, written in square brackets after its name — the same bracket notation collections (§13) already use for their own element types: `list[T]` is simply the built-in list mold, generic over T.

```
mold_decl   ::= 'mold' IDENT [ '[' type_params ']' ] '{' { field } '}'
task_decl   ::= 'task' IDENT [ '[' type_params ']' ] '(' [ params ] ')' [ ':' TYPE ] ':' block
type_params ::= type_param { ',' type_param }
type_param  ::= IDENT [ ':' bond_list ]
```

```
mold Box[T] {
    flux value: T
}

task max[T: Comparable](a: T, b: T): T
    when a.compare(b) >= 0:
        give a
    fallback:
        give b
```

A type parameter may be bounded by one or more bonds (§12), restricting it to types implementing the required behavior — here, `Comparable`. An unbounded parameter accepts any type.

4.6 Optional values: none & option[T]

`option[T]` is a built-in generic type, equivalent to declaring `alloy Option[T] { Some(T), None }` and making it implicitly available everywhere. It exists as a built-in because “a value that might be absent” is common enough to deserve one standard name rather than a different ad hoc convention in every codebase.

```
task find(nums: list[i32], target: i32): option[i32]
    each n in nums:
        when n == target:
            give Some(n)
    give None

match find(scores, 92):
    is Some(v):
        gen(v)
    is None:
        gen("not found")
```

5. Bindings & Mutability

```
binding_decl ::= ( 'lock' | 'flux' ) pattern [ ':' TYPE ] '=' expression
               | 'flux' IDENT [ ':' TYPE ]        # declared, assigned later
pattern      ::= IDENT | '(' IDENT { ',' IDENT } ')'
```

Keyword	Meaning
`lock`	Declares an immutable binding. Reassignment after the initial binding is a compile error.
`flux`	Declares a mutable binding (also the default — see below). May be declared without an initializer and assigned later.

A binding with no `lock`/`flux` keyword is a specification default for `flux`: bindings are mutable unless declared `lock`, matching the language's own worked examples (`x = 15` later reassigned as `x = x + 1` with no intervening declaration).

5.1 Destructuring

A tuple (§13.3) may be destructured directly on a binding's left-hand side; each name receives the corresponding element, and the pattern's arity must match the tuple's exactly.

```
lock (a, b) = pair
```

6. Expressions & Operators

Category	Operators
Arithmetic	`+ - * / %`
Comparison	`== != < > <= >=`
Logical	`and or not`
Bitwise	`& | ^ ~ << >>`
Assignment	`= += -= *= /=`
Member access	`.` (e.g. `p.x`)
Indexing	`buf[i]`

A mold implementing the stdlib-defined `Addable` bond (or its siblings for the other arithmetic and comparison operators) may be used directly with the corresponding operator — overloading is bond-mediated (§12), not a separate declaration, so an overloaded operator is always discoverable as an ordinary method.

7. Functions

7.1 Named functions

```
task_decl ::= 'task' IDENT '(' [ params ] ')' [ ':' TYPE ] ':' block
params    ::= param { ',' param }
param     ::= IDENT ':' TYPE [ '=' expression ]
            | '...' IDENT ':' TYPE
```

```
task connect(host: str, port: i32 = 8080): state
    ...

connect(host: "db.internal", port: 9090)   # named arguments, any order
connect("db.internal")                      # port defaults to 8080

task sum(...nums: i32): i32
    flux total = 0
    each n in nums:
        total += n
    give total
```

Keyword	Meaning
`task`	Declares a function.
`give`	Returns a value. A task with no give on some path implicitly gives no value.

A parameter with a default value may be omitted by the caller. A parameter prefixed with `...` collects any remaining positional arguments into a `list`; at most one variadic parameter is permitted, and it must be last.

7.2 Closures & first-class functions

A task value — named or anonymous — may be bound to a variable, passed as an argument, or returned from another task. A function value's type is written `(ParamTypes) -> ReturnType`.

```
closure ::= '(' [ params ] ')' '->' expression
```

```
lock double = (n: i32) -> n * 2
task apply_twice(f: (i32) -> i32, x: i32): i32
    give f(f(x))
gen(apply_twice(double, 5))     # 20
```

A closure needing more than one statement should be written as a nested, named task and referenced by name, since tasks are first-class.

8. Control Flow

8.1 Branching

```
when condition:
    ...
shift other_condition:
    ...
fallback:
    ...
```

Keyword	Role
`when`	if
`shift`	else if — may repeat
`fallback`	else — at most one, always last

8.2 Loops

Form	Role	Checks
`span condition:`	while	before each iteration
`cycle:`	infinite loop	never — exits only via escape
`scan x in buf:`	for — declares x over buf	once per element
`each y in buf:`	for-each, via peek or an Iterable bond (§12)	once per element
`repeat: ... until condition:`	do-while	after each iteration

`escape` exits the nearest loop (break); `skip` moves to its next iteration (continue).

8.3 Dispatch & pattern matching

```
choose x:
    is 1:
        ...
    fallback:
        ...

match shape:
    is Circle(r):
        ...
    is Square(side):
        ...
```

Keyword	Role
`choose`	Value-based dispatch (switch) over its argument.
`is`	One arm, under choose or match; under match, may bind an alloy variant's fields (§11).
`match`	Structural pattern dispatch, typically over an alloy.

Keyword	Role
`fallback`	Default arm. Required under choose unless exhaustive; required under match unless every alloy variant has an is arm.

9. Data Modeling: mold & forge

```
mold_decl  ::= 'mold' IDENT [ '[' type_params ']' ] '{' { field } '}'
field      ::= ( 'lock' | 'flux' ) IDENT ':' TYPE [ ';' ]

forge_decl ::= 'forge' IDENT [ ':' bond_list ] '{' { task_decl } '}'
```

```
mold Point {
    flux x: i32
    flux y: i32
}
forge Point {
    task init(a: i32, b: i32):
        x = a
        y = b
    task magnitude(): i32
        give x * x + y * y
}
```

Keyword	Role
`mold`	Declares a type's data layout. Brace-delimited (§2.5).
`forge`	Attaches tasks to a mold as methods, and optionally declares which bonds (§12) it satisfies. At most one forge block per mold.

Inside forge, an unqualified field name refers to the instance the method was called on (no explicit `self` in this version of the specification). A `lock` binding declared directly inside a forge block, outside any task, is a type-level constant — Dhar's equivalent of Java's `static final` fields, without a separate keyword.

9.1 Instantiation

An instance is created directly, with a struct literal listing every field (`Point { x: 1, y: 2 }`), or, when forge defines a task named `init`, by calling `Point.init(args)`.

10. Object-Oriented, Functional & Multi-Paradigm Design

Dhar's five target paradigms (Spec0 §2.2) are not five separate feature sets. Procedural code is the default; the sections below show how object-oriented, functional, logical, and symbolic styles are reached from the same small set of primitives, combined rather than duplicated.

10.1 Composition over inheritance

Dhar has no inheritance keyword and no extends relation between mold types. A type needing another's fields and behavior embeds it as a field instead, avoiding the fragile-base-class and diamond-inheritance problems classical inheritance is prone to. Polymorphism without inheritance is recovered explicitly through bond (§12), not given up.

```
mold Animal { flux name: str }
forge Animal {
    task speak(): str
        give name + " makes a sound"
}
mold Dog { flux base: Animal }
```

```
forge Dog: Speaker {
    task init(n: str):
        base = Animal { name: n }
    task speak(): str
        give base.name + " says woof"
}
```

10.2 Methods as first-class values

A bound method is itself a function value per §7.2, referenceable without being called.

```
lock rex = Dog.init("Rex")
lock greet = rex.speak
gen(greet())
```

10.3 Combining OOP and functional style

Because forge methods are ordinary function values and molds compose by embedding, an object-oriented data model and a functional pipeline read as one style.

```
lock dogs = [Dog.init("Rex"), Dog.init("Bo"), Dog.init("Zuri")]
each d in dogs:
    gen(d.speak())
```

10.4 Logical programming

No dedicated logic-programming keywords. Structural pattern matching (§8.3), collections (§13), and recursion are sufficient to write relations and queries directly, the way Scala or F# support a logic-programming style without a separate Prolog-like subsystem.

```
mold Fact { lock parent: str; lock child: str }

task is_grandparent(facts: list[Fact], a: str, c: str): bool
    each f1 in facts:
        when f1.parent == a:
            each f2 in facts:
                when f2.parent == f1.child and f2.child == c:
                    give true
    give false
```

10.5 Symbolic programming

Symbolic programming — representing code or expressions as data — now uses `alloy` (§11) rather than wild: the set of expression shapes is known in advance, so the type-safe, exhaustively-checked tool is the right one. (An earlier draft of this section used wild for this example; alloy is strictly better here and wild is reserved for genuinely unknown-until-runtime shapes, per §4.4.)

```
alloy Expr {
    Num(f64)
    Add(own Expr, own Expr)
    Mul(own Expr, own Expr)
}
task evaluate(e: Expr): f64
```

```
match e:
    is Num(v):
        give v
    is Add(l, r):
        give evaluate(l) + evaluate(r)
    is Mul(l, r):
        give evaluate(l) * evaluate(r)
```

Neither logical nor symbolic programming required a new keyword beyond alloy itself, which earns its place independently (§11). This is a deliberate specification stance: Dhar's multi-paradigm claim rests on a small orthogonal core composing well.

11. Sum Types: alloy

Where `mold` declares a product type (a value with all of its fields at once), `alloy` declares a sum type: a value that is exactly one of a fixed, named set of variants — Dhar's answer to Rust's enum. The name continues the material metaphor deliberately: an alloy is one thing actually made from a defined set of possible base components.

```
alloy_decl ::= 'alloy' IDENT [ '[' type_params ']' ] '{' { variant } '}'
variant    ::= IDENT [ '(' TYPE { ',' TYPE } ')' ]
```

```
alloy Shape {
    Circle(f64)
    Square(f64)
    Triangle(f64, f64, f64)
}

task area(s: Shape): f64
    match s:
        is Circle(r):
            give 3.14159 * r * r
        is Square(side):
            give side * side
        is Triangle(a, b, c):
            lock p = (a + b + c) / 2.0
            give (p * (p - a) * (p - b) * (p - c))
```

A `match` over an alloy **must** be exhaustive: every variant needs an is arm, or a fallback arm must be present. This is checked at compile time, closing the same class of bug `enforce` (§16) guards against at runtime — a forgotten case.

12. Interfaces: bond

bond declares a contract — a set of task signatures with no bodies — that a forge block may declare it satisfies. It is Dhar's mechanism for the classical-OOP polymorphism composition-over-inheritance (§10.1) deliberately gave up, recovered without a class hierarchy.

```
bond_decl ::= 'bond' IDENT '{' { signature } '}'
signature  ::= 'task' IDENT '(' [ params ] ')' [ ':' TYPE ]
```

```
bond Speaker {
    task speak(): str
}

forge Dog: Speaker {
    task speak(): str
        give name + " says woof"
}

task announce(s: bond Speaker):
    gen(s.speak())
```

A forge block declaring `: Speaker` **must** define every signature Speaker lists, checked at compile time — satisfaction is explicit, not structural, matching Dhar's general preference for checkable declarations over inferred ones. A parameter, field, or collection element typed `bond X` accepts any value whose forge declares `: X`,

dispatched dynamically. A type implementing the stdlib-defined `Iterable` bond may be used as the source of an `each` loop (§8.2), the same way a list can.

13. Collections

Five core collection forms. Each has literal syntax and a type name; none require a new reserved word.

13.1 Arrays

```
lock primes: [i32; 5] = [2, 3, 5, 7, 11]
gen(primes[2])          # 5, bounds-checked
```

13.2 Lists

```
scores = [98, 87, 92]        # list[i32], inferred
scores.push(75)
```

13.3 Tuples

```
pair = (1, "Dhar")            # (i32, str), inferred
gen(pair.0)
```

13.4 Sets

```
unique = {1, 2, 2, 3}         # set[i32] -- duplicates collapse
```

13.5 Dictionaries

```
ages = {"Ada": 36, "Grace": 85}     # dict[str, i32]
```

13.6 Comprehensions

A collection literal may generate its elements from a loop and an optional filter, reusing §8's own keywords rather than introducing separate comprehension syntax.

```
comprehension ::= '[' expression 'each' IDENT 'in' expression [ 'when' expression ] ']'
```

```
evens   = [n each n in scores when n % 2 == 0]
squares = [n * n each n in [1, 2, 3, 4]]
```

13.7 Stack, Queue & Deque

Not core-language literals; standard-library mold types built on list, generic over element type (§4.5), specified with Gearing Dhar (Stage-delta).

```
pull collections
flux s: collections.Stack[i32] = collections.Stack.init()
s.push(1)
lock top = s.pop()
```

14. Memory Model & References

14.1 Ownership

Every heap-resident value has exactly one owning binding at a time. Dhar combines compile-time ownership tracking with arena-style grouped deallocation; the compiler inserts the corresponding deallocation, with no manual free operation in the surface language. Unlike Rust, this tracking requires no lifetime annotations from the programmer — a deliberate simplification for the Python-class-simplicity commitment (Spec0 §2.2), traded against the compiler doing more inference work.

14.2 References: view & grab

Keyword	Role
`view T`	Read-only, non-owning reference. May not mutate the referent.
`grab T`	Mutable, non-owning reference. At most one live grab, and never alongside a live view of the same value.

14.3 Owned pointers: own

`own T` is a uniquely-owned, heap-allocated, nullable pointer to a value of type T — nullable specifically to make recursive data structures possible without infinite size. Checked the same way an option is: `is None` / `is Some(v)`. Unlike a general `option[T]`, own T's absence case costs no memory beyond the pointer itself — a specification note for Stage-gamma: this is expected to compile to a null-pointer check, not a separate tag byte, the same optimization Rust applies to `Option<Box<T>>`.

```
mold Node[T] {
    flux value: T
    flux next: own Node[T]
}

task push_front[T](head: own Node[T], v: T): own Node[T]
    give Node[T] { value: v, next: head }
```

14.4 Raw memory & volatile

`raw[N]` declares a fixed-size, uninterpreted buffer of N bytes with pointer-like semantics. `peek`/`poke` are its explicit read/write primitives, since a raw buffer has no element type for the compiler to infer a width from.

```
address: raw[4096]
poke(address, 0, 255)
lock v = peek(address, 0)
```

A raw[N] declaration may be qualified `volatile`: every peek/poke against it has an observable side effect outside the program's own memory model and **must not** be reordered, cached, or eliminated, even if it appears redundant. Required for memory-mapped hardware registers (Spec0 §5); this is a correctness requirement, not a performance hint.

```
control_register: volatile raw[4]
poke(control_register, 0, 1)   # must actually happen
```

Every indexing operation, typed or raw, is bounds-checked; an out-of-range index **must** produce a safe, diagnosable failure and **must not** silently touch adjacent memory.

15. Concurrency: spark & draw

A task marked `spark` runs asynchronously: calling it returns a handle to its eventual result rather than blocking. `draw` suspends the calling task until that result is available, and unwraps it. Modeled on async/await as implemented in Rust and JavaScript, chosen over a bare-threads model so concurrency is visible in a task's signature rather than hidden.

```
spark task fetch(url: str): str
    ...

task core():
    lock pending = fetch("https://example.com")
    ...                        # other work runs while fetch is in flight
    lock body = draw pending
    gen(body)
```

A `grab` reference (§14.2) **must not** be captured by a `spark` task while the capturing scope also holds it live, and this exclusivity rule applies across task boundaries, not only within a single sequential scope. This closes data races by the same construction that already closes ordinary aliasing bugs, rather than as a separate concurrency-specific mechanism — a value cannot be mutated from two places at once, whether those two places are two lines of sequential code or two concurrently running tasks.

16. Error Handling

16.1 The state type

`state` is a built-in result type: `Success` or `Error`. A task whose outcome can fail should return state rather than a sentinel value.

16.2 Handling a state

Keyword	Role
`trap`	Handles a state's Error case explicitly.
`enforce`	Asserts a condition; terminates the program if false. For invariants whose failure means internal state can no longer be trusted.

16.3 Structured exceptions

```
attempt:
    risky_operation()
rescue:
    ...
ensure:
    cleanup()
```

Keyword	Role
`attempt`	try
`rescue`	catch

Keyword	Role
`ensure`	finally
`raise`	throw

state/trap is for expected, typed failure a caller must handle every time; attempt/rescue is for exceptional conditions unwinding across call frames. A task's signature must say which mechanism it uses; the two must not be mixed for the same failure within one task.

17. Modules & Visibility

Keyword	Role
`pull`	Imports another module by name.
`expose`	Marks a task, mold, or binding visible outside its file. File-private by default.

Privacy is file-level, not per-type — a deliberate match to Rust's module-level privacy rather than Java's class-level granularity.

18. Foreign Function Interface: foreign

`foreign` declares a task with no Dhar body, implemented externally and linked at build time via the target platform's C ABI. It exists to make calling existing C libraries possible, a practical necessity for adoption — and it is, deliberately, the only place Dhar's independence commitment (Spec0 §2.2) is allowed to bend. A foreign-declared function is external by definition; the Canon does not pretend otherwise.

```
foreign_decl ::= 'foreign' 'task' IDENT '(' [ params ] ')' [ ':' TYPE ]
```

```
foreign task c_malloc(size: u64): own raw[0]
foreign task c_free(ptr: own raw[0]):
```

19. Inline Assembly: asm

`asm` inserts raw instructions for the current backend's target architecture verbatim into the generated output. Each string is one instruction line. asm is inherently non-portable, and is a narrow escape hatch for the handful of operations (hardware counters, atomics the language doesn't yet expose, memory barriers) with no other expression in Dhar — not a general-purpose alternative to writing Dhar.

```
asm_stmt ::= 'asm' STRING { STRING }
```

```
task read_cycle_count(): u64
    asm "rdtsc"
        "shl rdx, 32"
        "or rax, rdx"
```

20. System Interface

Keyword	Role
`sys`	Raw system call; arguments map to registers per the target's calling convention.
`sysret`	Captures the return value of the most recent sys call.
`gen`	Portable print/output, desugaring to the appropriate sys call.

```
gen("hello, dhar")          # portable, concise
sys 1, 1, msg, len          # explicit, platform-specific
```

21. Grammar Summary

```
program     ::= { mold_decl | forge_decl | alloy_decl | bond_decl
                | task_decl | foreign_decl }
block       ::= INDENT { statement } DEDENT
statement   ::= binding_decl | assignment | task_call | branch | loop
              | dispatch | 'give' [ expression ] | 'escape' | 'skip'
              | 'pull' IDENT | error_stmt | asm_stmt
branch      ::= 'when' expression ':' block
                { 'shift' expression ':' block } [ 'fallback' ':' block ]
loop        ::= 'span' expression ':' block
              | 'cycle' ':' block
              | 'scan' IDENT 'in' expression ':' block
              | 'each' IDENT 'in' expression ':' block
              | 'repeat' ':' block 'until' expression ':' block
dispatch    ::= ( 'choose' | 'match' ) expression ':'
                { 'is' pattern ':' block } [ 'fallback' ':' block ]
mold_decl   ::= 'mold' IDENT [ '[' type_params ']' ] '{' { field } '}'
field       ::= ( 'lock' | 'flux' ) IDENT ':' TYPE [ ';' ]
forge_decl  ::= 'forge' IDENT [ ':' bond_list ] '{' { task_decl } '}'
alloy_decl  ::= 'alloy' IDENT [ '[' type_params ']' ] '{' { variant } '}'
variant     ::= IDENT [ '(' TYPE { ',' TYPE } ')' ]
bond_decl   ::= 'bond' IDENT '{' { signature } '}'
foreign_decl::= 'foreign' 'task' IDENT '(' [ params ] ')' [ ':' TYPE ]
asm_stmt    ::= 'asm' STRING { STRING }
closure     ::= '(' [ params ] ')' '->' expression
TYPE        ::= PRIMITIVE | IDENT | 'wild'
              | 'view' TYPE | 'grab' TYPE | 'own' TYPE | 'bond' IDENT
              | '[' TYPE ';' INTEGER ']' | '[' TYPE ']'
              | '(' TYPE { ',' TYPE } ')'
              | 'set' '[' TYPE ']' | 'dict' '[' TYPE ',' TYPE ']'
              | 'option' '[' TYPE ']'
              | '(' [ TYPE { ',' TYPE } ] ')' '->' TYPE
error_stmt  ::= 'attempt' ':' block
                [ 'rescue' ':' block ] [ 'ensure' ':' block ]
              | 'raise' expression | 'trap' expression [ ':' block ]
              | 'enforce' expression
```

Appendix A — Reserved Words & Cross-Language Equivalents

For readers arriving from another language. "—" marks no direct equivalent.

Keyword	Meaning	C / C++	Rust	Python	Java	JavaScript
`task`	Function decl.	function decl.	fn	def	method decl.	function
`give`	Return a value	return	return	return	return	return
`lock`	Immutable binding	const	let	— (convention)	final	const
`flux`	Mutable binding (default)	plain decl.	let mut	plain assignment	plain decl.	let
`wild`	Dynamic-typed binding	void* / std::any	dyn Any / enum	— (native default)	Object / var	— (native default)
`when`	if	if	if	if	if	if
`shift`	else if	else if	else if	elif	else if	else if
`fallback`	else / default arm	else / default	else / _	else	else / default	else / default
`span`	while	while	while	while	while	while
`cycle`	infinite loop	for(;;)	loop	while True	while(true)	while(true)
`scan…in`	for	for(…)	for x in	for x in	for(…)	for(…)
`each…in`	for-each	range-for	.iter() for	for x in	enhanced for	for…of
`in`	Loop range/collection binder	: (range-for)	in	in	:	of / in
`escape`	break	break	break	break	break	break
`skip`	continue	continue	continue	continue	continue	continue
`repeat…until`	do-while	do{}while()	loop + break	while True + break	do{}while()	do{}while()
`until`	do-while condition	while() part	—	—	while() part	while() part
`choose`	switch	switch	match	match (3.10+)	switch	switch
`is`	case / pattern arm	case	match arm	case	case	case
`match`	Pattern dispatch	— (nested if)	match	match (3.10+)	switch (21+)	—
`mold`	Struct/type layout	struct / class	struct	class	class	class
`forge`	Method-attachment block	methods in class	impl	methods in class	methods in class	methods in class

Keyword	Meaning	C / C++	Rust	Python	Java	JavaScript
`alloy`	Sum type / tagged union	union (untagged) / variant	enum	— (classes)	— (sealed, 17+)	— (tagged object)
`bond`	Trait / interface	— (abstract class)	trait	— (Protocol)	interface	— (duck typing)
`view`	Read-only reference	const T&	&T	—	—	—
`grab`	Mutable reference	T&	&mut T	—	—	—
`own`	Owned heap pointer	unique_ptr	Box<T>	— (native default)	— (native default)	— (native default)
`peek`	Read raw memory	*ptr	unsafe *ptr	—	— (Unsafe)	DataView.get*
`poke`	Write raw memory	*ptr = v	unsafe *ptr = v	—	— (Unsafe)	DataView.set*
`state`	Success|Error result type	— (errno)	Result<T,E>	— (exceptions)	— (Optional)	— (exceptions)
`trap`	Handle a state	—	match / ? on Result	—	—	—
`enforce`	Critical assertion	assert()	assert!()	assert	assert	— (console.assert)
`attempt`	try	try	— (no exceptions)	try	try	try
`rescue`	catch	catch	—	except	catch	catch
`ensure`	finally	— (RAII)	— (Drop)	finally	finally	finally
`raise`	throw	throw	— (panic!)	raise	throw	throw
`pull`	Import a module	#include	use	import	import	import
`expose`	Mark public	public: (class)	pub	— (convention)	public	export
`spark`	Mark a task async	— (threads)	async fn	async def	— (CompletableFuture)	async function
`draw`	Await an async result	—	.await	await	.get() (Future)	await
`foreign`	FFI declaration	extern “C”	extern “C”	ctypes.CDLL	native (JNI)	— (WASM FFI)
`asm`	Inline assembly	asm / __asm__	asm!()	— (ctypes)	— (JNI)	—
`volatile`	Non-optimizable memory access	volatile	read_volatile	—	volatile (field)	—
`sys`	Raw system call	syscall()	libc::syscall	— (os module)	—	—
`sysret`	Capture syscall return	syscall() return	same	—	—	—
`gen`	Portable print/output	printf	println!	print	System.out.println	console.log

Spec1 — Language Specification · v3.1 · September 8, 2026 · Ratified. Revision log: v1.0 — first edition, 37 keywords. v2.0 — brace-delimited mold/forge, wild, closures, multi-paradigm section, collections; 38 keywords. v3.0 — added generics (§4.5), option/none (§4.6), destructuring (§5.1), bitwise operators (§6), default/named/variadic parameters (§7.1), alloy sum types (§11), bond interfaces (§12), comprehensions (§13.6), own owned pointers (§14.3), volatile (§14.4), spark/draw concurrency (§15), foreign FFI (§18), asm inline assembly (§19), and f-string interpolation (§2.4); rewrote §10.5 to use alloy instead of wild; 46 keywords. v3.1 — §15 now states explicitly that view/grab exclusivity applies across spark/draw task boundaries, closing data races by the same construction as ordinary aliasing bugs (added for Tempering Dhar, Volume III).